

# **RESOLUCIÓN DE PROBLEMAS Y ALGORITMOS**

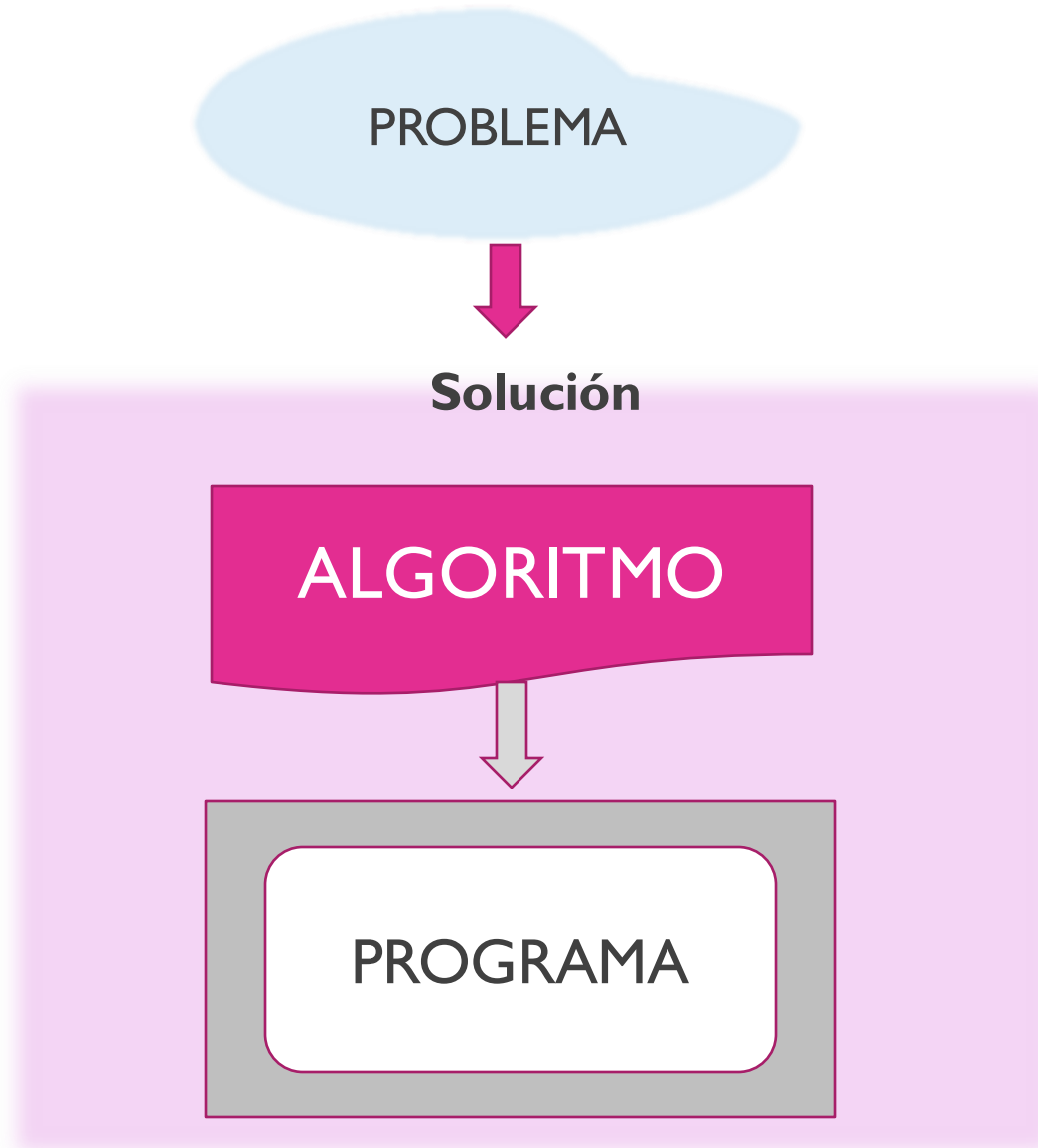
**Dra. Jessica Andrea Carballido**

**2**

**Dpto. de Ciencias e Ingeniería de la Computación  
UNIVERSIDAD NACIONAL DEL SUR**

**CLASE 2**

# Problemas, Algoritmos y Programas



# Paradigmas de Programación

## Imperativos

### Estructurado

Dividido en bloques

Secuencia, selección e iteración

### Orientado a Objetos

Abstracción de datos

Encapsulamiento de datos y comportamientos en objetos

Paso de mensajes entre objetos

## Declarativos

### Funcional

Las funciones son elementos de primer orden

Uso del cálculo lambda

### Lógico

Los programas están contruidos de hechos, predicados y relaciones

acciones

datos



# PARADIGMAS DE PROGRAMACIÓN



**Diferentes formas de programar**



**Resolver el problema de diferentes formas**



**Programación estructurada**

- Programación secuencial
- Usa ciclos y condicionales



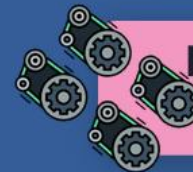
**Programación Funcional**

- Divide el programa en pequeños objetos.
- Las tareas son ejecutadas por funciones.



**Programación Orientada a objetos**

- Divide un sistema en partes (objetos)
- Se comunican entre ellos



**Programación Reactiva**

- Observa flujos de datos asíncronos
- Reacciona frente a los cambios



# Problemas, Algoritmos y Programas

Un algoritmo es una secuencia de instrucciones, comprensibles para quien tenga que ejecutarlas, que permiten resolver un problema.

Un programa procedural es un algoritmo escrito en un lenguaje de programación imperativo.

Nos interesa escribir **algoritmos generales** para implementarlos en programas que **terminen en un tiempo finito**, **resuelvan correctamente el problema propuesto**, **sean eficientes** y **estén bien estructurados**.

Resuelven  
una familia  
de problemas

# Programas y Lenguajes

Un **lenguaje de programación** es una notación **formal** que puede ser interpretada por una computadora.

```
[*]----- programa.pas -----
program programa;

var num1, num2 : integer;

begin
    write('Ingresa un numero ');
    readln(num1);
    write('Ingresa otro numero ');
    readln(num2);

    if num1 > num2 then
        write(num1, ' es mayor a ', num2);
    else
        write(num1, ' es menor a ', num2);
    end.
end.
```

# Programas y Lenguajes

Los humanos nos expresamos en un lenguaje **natural**.

En un lenguaje natural una misma palabra  
puede estar asociada a distintos significados  
y una misma oración  
puede interpretarse de dos o más maneras diferentes.

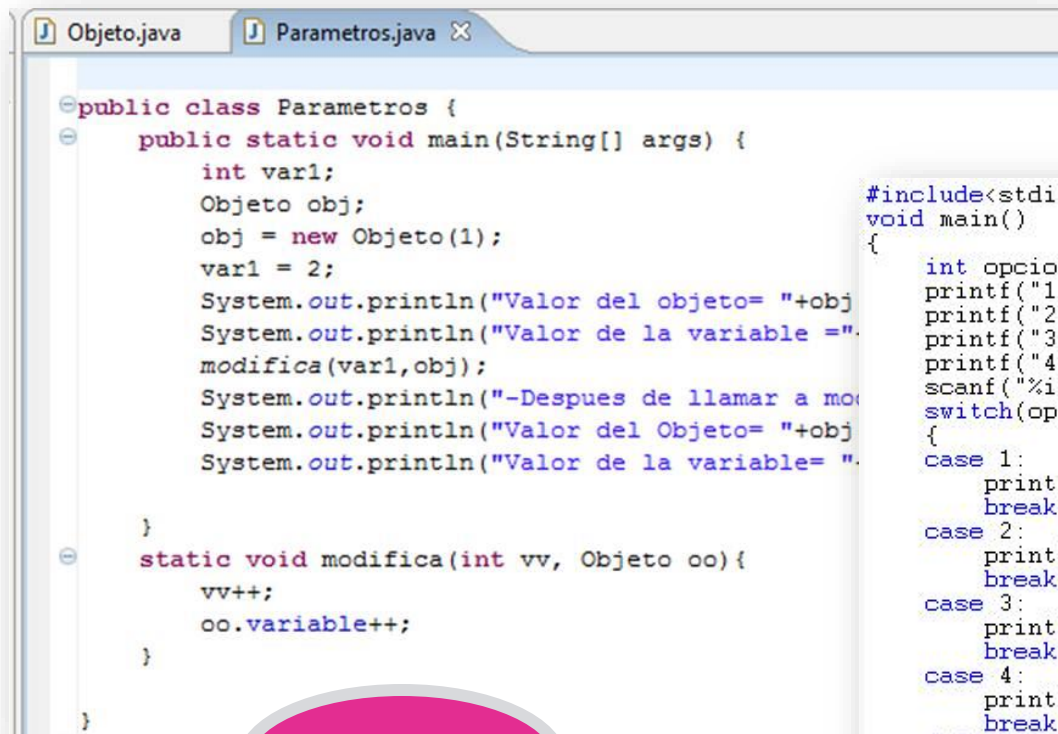
Un lenguaje de programación es una notación **formal**  
porque tiene una **sintaxis estricta** y una **semántica precisa**.

- La sintaxis se refiere a la **forma** de los programas escritos en el lenguaje.
- La semántica se refiere al **significado**.



# Programas y Lenguajes

Cada lenguaje de programación tiene sus propias **reglas sintácticas** que determinan la forma de las instrucciones y la estructura general del programa.



```
Objeto.java Parametros.java X
public class Parametros {
    public static void main(String[] args) {
        int var1;
        Objeto obj;
        obj = new Objeto(1);
        var1 = 2;
        System.out.println("Valor del objeto= "+obj);
        System.out.println("Valor de la variable = "+var1);
        modifica(var1,obj);
        System.out.println("-Despues de llamar a modifica");
        System.out.println("Valor del Objeto= "+obj);
        System.out.println("Valor de la variable= "+var1);
    }
    static void modifica(int vv, Objeto oo){
        vv++;
        oo.variable++;
    }
}
```

JAVA

```
#include<stdio.h>
void main()
{
    int opcion;
    printf("1. Capital de Argentina\n");
    printf("2. Capital de España\n");
    printf("3. 10000+58000 = ?\n");
    printf("4. Capital de Uruguay\n");
    scanf("%i",&opcion);
    switch(opcion)
    {
        case 1:
            printf("\n\nBuenos Aires");
            break;
        case 2:
            printf("\n\nMadrid");
            break;
        case 3:
            printf("\n\n68000");
            break;
        case 4:
            printf("\n\nMontevideo");
            break;
        default:
            printf("\n\nOpcion erronea. Intenta de nuevo.");
    }
}
```

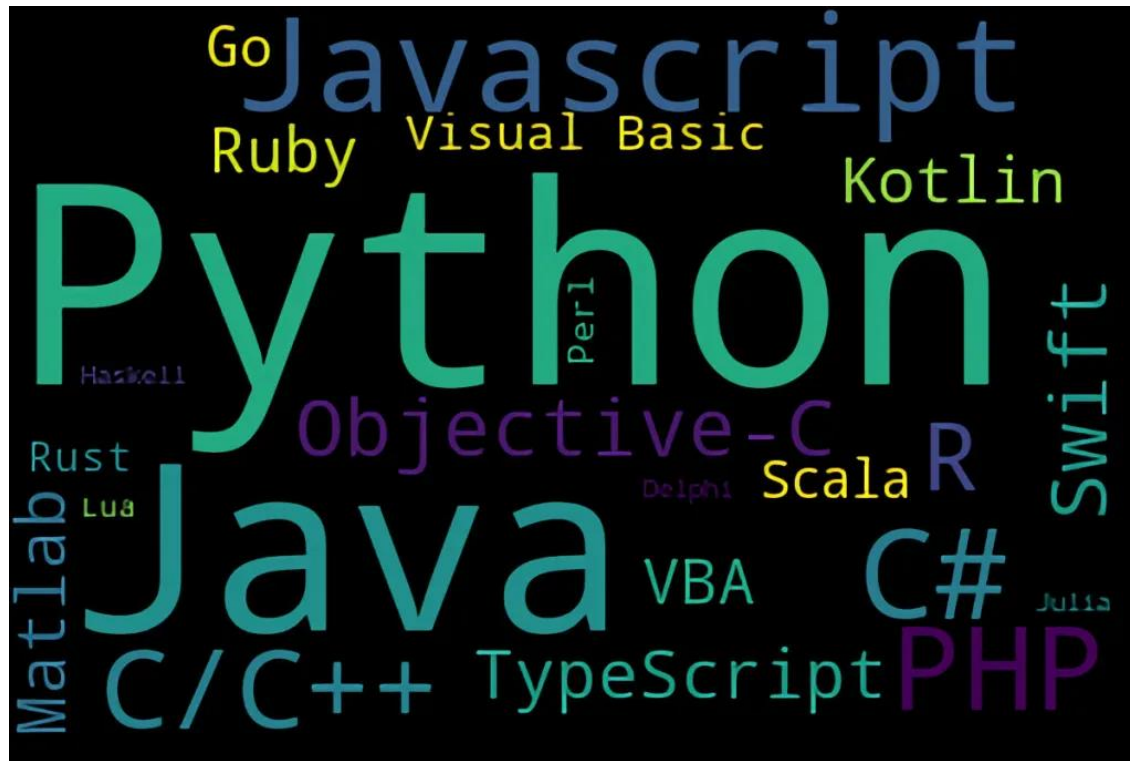
C



# Programas y Lenguajes

No existe un lenguaje de programación *ideal*,  
cada dominio de aplicación impone restricciones diferentes.

Una capacidad fundamental para un **profesional** de la disciplina  
es justamente **aprender** de manera autónoma un nuevo lenguaje  
y elegir el más adecuado para una aplicación específica.



# El lenguaje Pascal

Pascal es un lenguaje de programación desarrollado por el profesor suizo *Niklaus Wirth* a finales de los años 60.

El objetivo fue crear un lenguaje que facilitara el **aprendizaje** de la programación.

Sin embargo en los años siguientes y hasta los '80, su uso excedió el ámbito académico para convertirse en una herramienta para el desarrollo de sistemas para el comercio, la industria, educación, etc.

*“Método disciplinado y elegante  
a la hora de programar.”*



# El lenguaje Pascal

Principales características:

**Claridad y legibilidad.** Se puede entender cuando se lee el código. Si un programa está claramente escrito, debe ser posible que otro programador siga la lógica sin esfuerzo (sin contar al autor original que lo ha escrito, sobretodo pasado un tiempo).

*“Su uso obliga al desarrollo de programas  
**bien organizados, escritos con claridad**  
y relativamente **libres de errores.**”*



# El lenguaje Pascal

En la actualidad Pascal no se utiliza para desarrollar sistemas comerciales pero se sigue usando para enseñar a programar.

Las **características** de Pascal que presentaremos en RPA están **incluidas** también en **muchos** de los **lenguajes** de programación que se usan para desarrollar software en la industria.

Asignación, expresiones, tipos elementales y estructuras de control condicionales e iterativas tienen una sintaxis y semántica similar a la de Java (lenguaje que usaremos en la materia que sigue y que sí es utilizado en la industria).



- Ej. (condicional.py)

```
q = 4
h = 5
if q < h :
    print "primer test pasado"
elif q == 4:
    print "q tiene valor 4"
else:
    print "segundo test pasado"
>>> python condicional.py
primer test pasado
```

- Operadores booleanos: "or," "and," "not"
- Operadores relacionales: ==, >, <, !=

# PYTHON

```
if a > b
    puts "a es mayor que b"
elsif a == b
    puts "a es igual a b"
else
    puts "b es mayor que a"
end
```

# RUBY

## Condicionales

# JAVA

```
public class EjemploCondicional1{
    public static void main(String args[]){
        int edadLuis=24;
        if(edadLuis>=18){
            System.out.println("Luis es mayor de edad y puede votar");
            System.out.println("Luis tiene "+edadLuis+" a"+(char)164+"os");
        }
        else{
            System.out.println("Luis no es mayor de edad y no puede votar");
            System.out.println("Luis tiene "+edadLuis+" a"+(char)164+"os");
        }
        System.out.println("FIN DE PROGRAMA");
    }
}
```

# El lenguaje Pascal: COMPILADOR

## Algoritmo

Algoritmo saludo

comienzo

Mostrar cartel: HOLA MUNDO

fin



## Programa (PASCAL)

Program saludo;

begin

    write("hola mundo");

End.

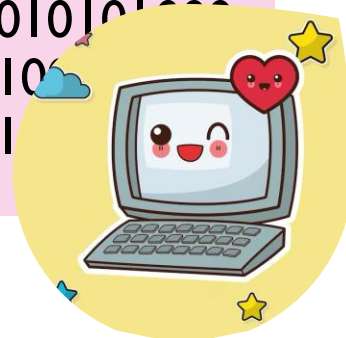
compilador

mostrarDato(**A**)  
mostrarCartel('HOY')  
mostrar('PERIMETRO: ', **P**)

write

*Primitiva que nos brinda el lenguaje*

00000101010000100010101010  
10100000011110001010101011  
01000001011110000010101000  
00010101000000100101010101  
00101001010101010101010101



# El lenguaje Pascal: COMPILADOR

Un **compilador** es un traductor que comprueba la validez y transforma un programa escrito en un lenguaje de programación (código fuente) a un programa en código máquina.

- 1ra etapa: ANÁLISIS

La compilación permite detectar errores sintácticos y algunos errores semánticos en el código fuente.

- 2da etapa: TRADUCCIÓN

Si el código fuente no tiene errores sintácticos ni semánticos, el compilador lo traduce a código objeto o máquina.





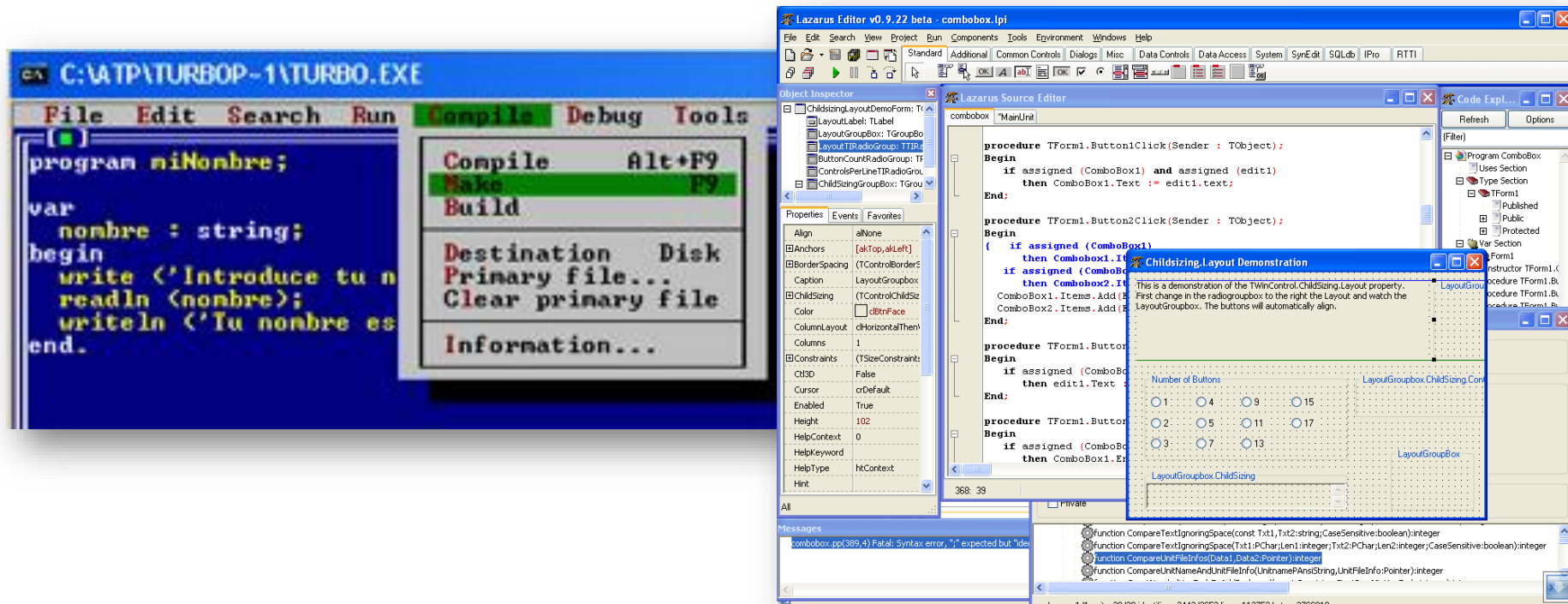
# El lenguaje Pascal

## Existen diferentes versiones de compiladores de Pascal

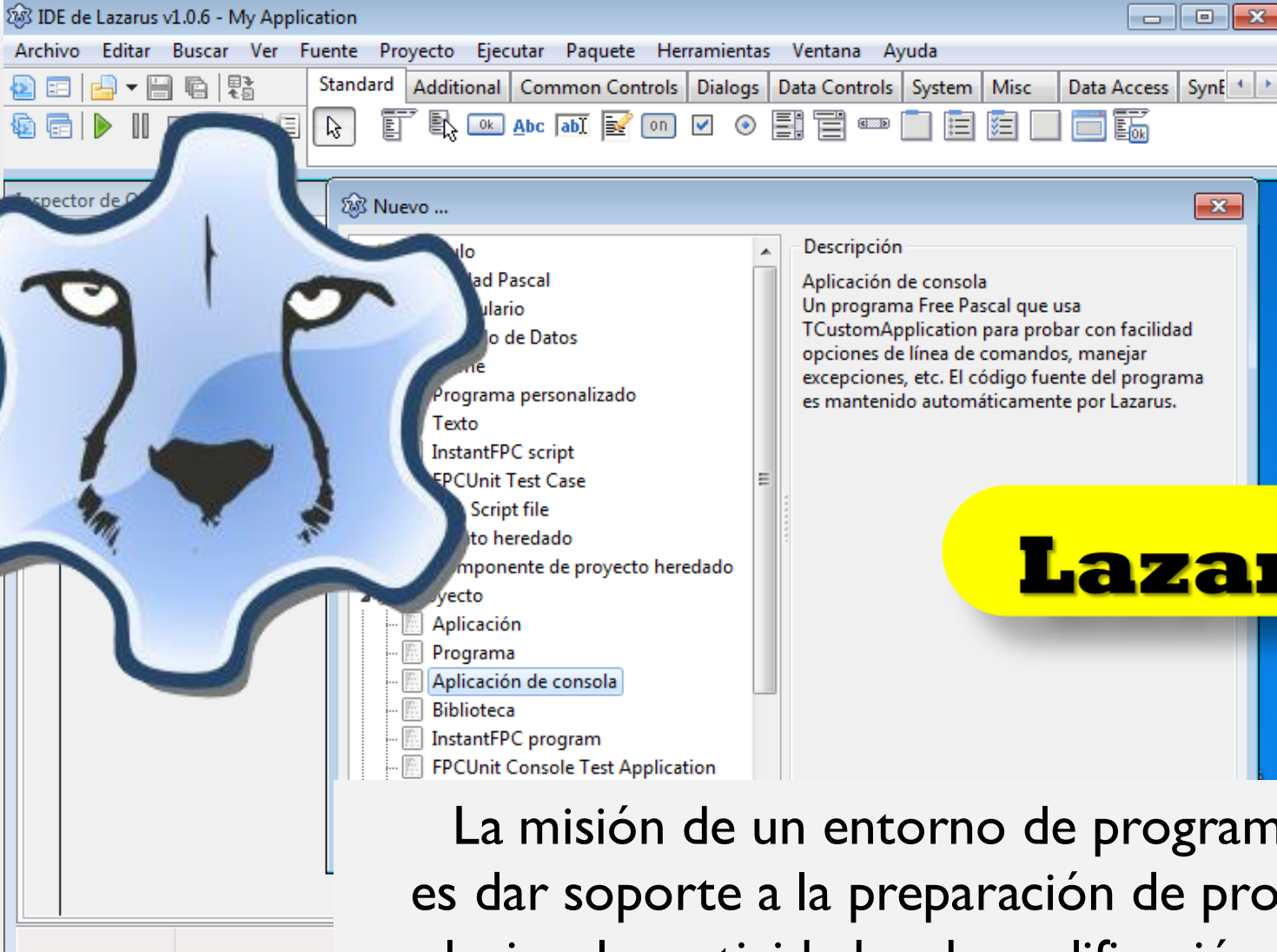
(Turbo Pascal, Free Pascal)

y distintos entornos de programación (Delphy, Geany, Lazarus)

que permiten **editar, compilar y ejecutar** programas.



# El entorno de programación integrado (IDE)



The image shows a screenshot of the Lazarus IDE v1.0.6 interface. The main window is titled "IDE de Lazarus v1.0.6 - My Application". It features a menu bar with options like Archivo, Editar, Buscar, Ver, Fuente, Proyecto, Ejecutar, Paquete, Herramientas, Ventana, and Ayuda. Below the menu bar is a toolbar with various icons for file operations, editing, and execution. A "Nuevo ..." (New...) dialog box is open, showing a list of project templates on the left and a description on the right. The templates include "Aplicación de consola", "Programa", "Biblioteca", "InstantFPC program", and "FPCUnit Console Test Application". The "Aplicación de consola" option is selected. The description on the right states: "Aplicación de consola. Un programa Free Pascal que usa TCustomApplication para probar con facilidad opciones de línea de comandos, manejar excepciones, etc. El código fuente del programa es mantenido automáticamente por Lazarus." A stylized, blue, multi-lobed face with large eyes and a mustache is overlaid on the left side of the screenshot. A yellow rounded rectangle with the word "Lazarus" in bold black text is positioned on the right side of the screenshot.

**Lazarus**

La misión de un entorno de programación es dar soporte a la preparación de programas, es decir, a las actividades de codificación y pruebas.

## Estructura general de un algoritmo

Algoritmo ...

DE: **datos de entrada**

DS: **datos de salida**

..

**ACCIONES**

..

## Estructura general de un programa en Pascal

Program ..... ;

Var ..... **datos** ;

Begin

..... **ACCIONES** separadas por punto y coma

End.

**Pascal  
programming**

# Elementos de Pascal

Problema: Calcular y mostrar el área de un cuadrado a partir de la longitud del lado ingresada por el usuario.

Algoritmo Area De Un cuadrado

DE: lado (entero)

DS: area (entero)

$area \leftarrow lado * lado$

Resuelve un problema general,  
no una instancia particular  
de un problema



# Elementos de Pascal

Problema: Calcular y mostrar el área de un cuadrado a partir de la longitud de lado ingresada por el usuario.

Resuelve un problema general, no una instancia particular de un problema

```
program areaCuadrado;  
var lado, area: integer;  
begin
```

Leer dato  
de entrada

```
  write('Ingrese la longitud del lado:');  
  readln(lado);
```

Mostrar dato  
de salida.

```
  area := lado * lado;
```

```
  writeln('El área del cuadrado es ', area, 'cm.');
```

```
end.
```



# Elementos de Pascal

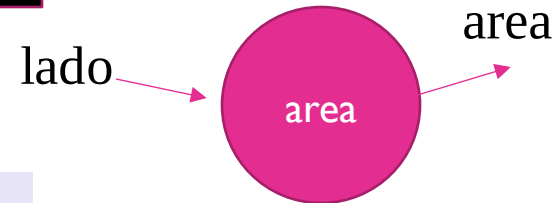
Lo ingresa el usuario

Cambian en  
cada  
ejecución

Ingrese la longitud del lado: 6  
El área del cuadrado es 36 cm.

Lo muestra el programa

**CONSOLA**



La comunicación con el usuario  
se realiza a través de la consola.

# Elementos de Pascal

```
program areaCuadrado;  
var lado, area: integer;  
begin
```

```
  write('Ingrese la longitud del lado:');
```

Leer dato  
de entrada

```
  readln(lado);
```

```
  area := lado * lado;
```

Mostrar dato  
de salida.

```
  writeln('El área del cuadrado es ', area, 'cm.');
```

```
end.
```

Algoritmo Area De Un cuadrado

DE: lado

DS: area

area ← lado \* lado

Pascal  
programming

## Acciones:

- writeln → **muestra** un mensaje (texto entre comillas) o el valor de un dato en la consola
- readln → **toma** un valor de la consola y lo almacena en el dato que esta entre paréntesis
- Asignación



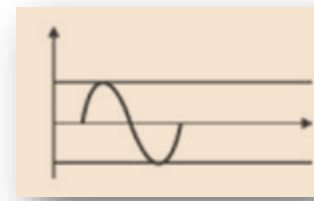
Prestar atención a:

- **Punto y coma** después de la primera línea  
“program P;”
- Punto y coma después de declarar variables de un tipo  
“var dato1, dato2: integer;”
- Sintaxis de la declaración de los datos
- Punto y coma después de cada acción
- **begin** y **end** encerrando las acciones.
- **Punto** solamente luego del end donde termina el programa.





# Variables y Constantes



Los **datos** de un programa están representados por **variables** o **constantes** y tienen asociado un nombre y un tipo.

El **tipo de un dato** determina el conjunto de valores y el conjunto de operadores que se aplican sobre estos valores.

Integer  
Real  
Char  
Boolean

El tipo de dato permite realizar **controles** para prevenir algunos errores.

Si en una expresión, un operador está asociado con operandos que no son compatibles con su tipo, el **compilador** reporta un error (**error semántico de compilación**).

Boolean  
Char

```

9  program Hello;
10 var a: integer;
11 begin
12
13     a := 5.1;
14
15 end.

```

- 5.1 es un valor real
- No puedo almacenar un valor real en un dato que fue declarado de tipo entero.

Compilation failed due to following error(s).

```

Free Pascal Compiler version 3.2.2+dfsg-9ubuntu1 [2022/04/11] for x86_64
Copyright (c) 1993-2021 by Florian Klaempfl and others
Target OS: Linux for x86-64
Compiling main.pas
main.pas(13,7) Error: Incompatible types: got "Extended" expected "SmallInt"
main.pas(20) Fatal: There were 1 errors compiling module, stopping
Fatal: Compilation aborted
Error: /usr/bin/ppcx64 returned an error exitcode

```

# Elementos de Pascal

Problema: Calcular y mostrar el área de un cuadrado a partir de la longitud de lado ingresada por el usuario.

```
program areaCuadrado;
```

```
var lado, area: integer;
```

```
begin
```

```
    write('Ingrese la longitud del lado:');
```

```
    readln(lado);
```

```
    area := lado * lado;
```

```
    writeln('El área del cuadrado es ', area, 'cm.');
```

```
end.
```

**LOS DATOS**

No se separan por DE y DS, se separan de acuerdo a su **tipo**.

El tipo integer brinda un subconjunto de los números enteros y un **conjunto de operaciones** definidas para este subconjunto de valores.

El subconjunto está definido por el rango  $[-\text{MaxInt}, \text{MaxInt}]$   
donde MAXINT es una constante predefinida.

Algunos operadores predefinidos del tipo integer son:

**+ - \* div mod sqr abs sqrt**

2.147.483.647

# MAXINT

# DEPENDE DE MUCHAS COSAS



# El tipo integer

tIPOS

Dadas las declaraciones

```
var x, y, M, N: integer;
```

Son válidas las expresiones:

$2+x*y$

$M \bmod N$

$\text{sqr}(x) \text{ div } 2$

$\text{abs}(M)$



```

program claseTIP05;
var a: integer;
begin
  a := -5;
  writeln ('a');
  writeln(a);
  writeln(sqr(a));
  writeln(2+a*1.5);
  writeln(abs(a));
end.

```



```

Compiling main.pas
Linking a.out
19 lines compiled, 0.0 sec
a
-5
25
-5.5000000000E+00
5

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

# El tipo real

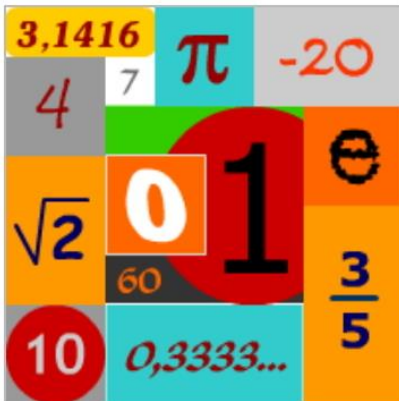
tipos

El tipo real incluye a un subconjunto de los números reales; aquellos que están dentro de un rango dado y con cierta precisión.

Algunos de los operadores predefinidos del tipo real:

**+ - \* / abs sqr sin cos sqrt**

Computan un valor de tipo real al aplicarse sobre un operando de tipo real.



**trunc round**

Se aplican sobre un argumento de tipo real y computan un número entero

# El tipo real

tipos

Dadas las declaraciones

`var v1, v2: real;`

Son válidas las expresiones:

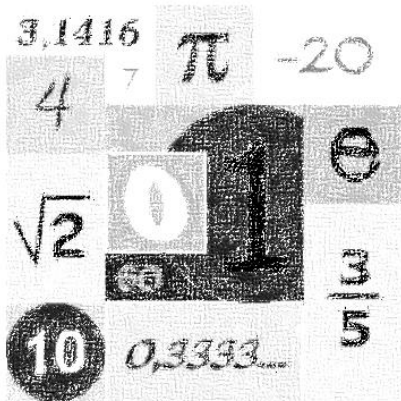
`2.1 + v1/v2`

`sqr(v1)`

`abs(v2-v1)`

`sqrt(v1) + sqrt(v2*v2)`

`sin(v1)`

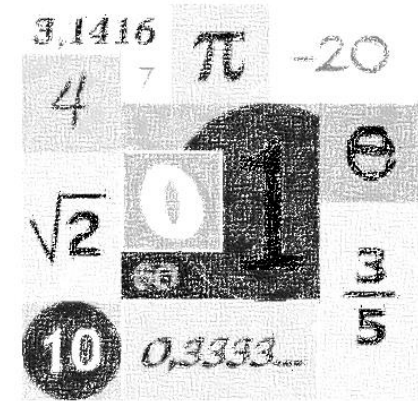




```

2 program Hello;
3 var a: integer;
4     b: real;
5 begin
6
7     a := 10;
8     b := 3.5;
9
10    writeln(a+b);
11    writeln(sqrt(a));
12    writeln(sqrt(b));
13    a := trunc(b);
14    writeln('b truncado: ', a);
15    a := round(b);
16    writeln('b redondeado: ', a);
17
18 end.

```



version 3.2.2+dfsg-9ubuntu1 [2022/04/11] for x86\_64  
 1 by Florian Klaempfl and others

```

Target OS: Linux for x86-64
Compiling main.pas
Linking a.out
20 lines compiled, 0.1 sec
1.3500000000000000E+001
3.16227766016837933197E+0000
1.8708286933869707E+000
b truncado: 3
b redondeado: 4

```

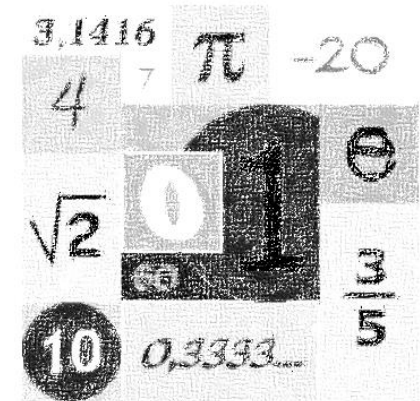
```

...Program finished with exit code 0
Press ENTER to exit console.

```

Para mostrar números reales con un formato más amigable:

```
program Decimales;  
var z: real;  
begin  
  z := 4389.42;  
  writeln(z);  
  writeln(z:0:2);  
end.
```



```
Free Pascal Compiler version 3.2.2+dfsg-32 [2024/01/05]  
Copyright (c) 1993-2021 by Florian Klaempfl and others  
Target OS: Linux for x86-64  
Compiling main.pas  
Linking a.out  
15 lines compiled, 0.0 sec  
  4.38942000000000001E+003  
4389.42  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

## Procedimiento `writeln`

- Escribe en la consola los argumentos que recibe, y a continuación, un salto de línea
- Para usar `:0:n` es necesario que el argumento del `write` sea real, no puede ser entero

## Especificar la anchura

- Se puede especificar la anchura de lo escrito, mediante el símbolo de dos puntos (:) y la cifra que indique la anchura 
- `anchura` es una expresión entera (literal, constante, variable o llamada a función) que especifica la anchura total del campo en que se escribe el valor 

## Ejemplo

```
write('Precio: '); write(Precio:0:2); 
```

- `writeln (valor:anchura:dígitos ...);`
- `dígitos` son los dígitos decimales de un número real
- `anchura` es el total de dígitos del número real contando parte entera, punto decimal y dígitos decimales

# El tipo boolean



El tipo boolean incluye solo a dos valores que son las constantes predefinidas *true* y *false*, y los tres operadores lógicos **or**, **and** y **not**.

El valor de las variables booleanas se puede imprimir pero no leer.

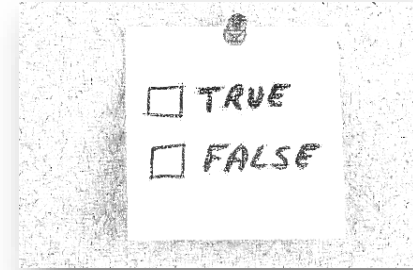
Los operadores relacionales  $<$ ,  $>$ ,  $=$ ,  $<>$ ,  $<=$ ,  $>=$  se aplican a operandos de tipo integer, real o char y computan un valor de tipo boolean

# El tipo boolean

```
const minimo = 10;  
      maximo = 20;  
var temp: integer;  
      seSuspende: boolean;  
...
```

```
seSuspende := (temp < minimo) or (temp > maximo);
```

*¿Cuanto vale seSuspende si temp = 10?*



Pascal no entiende:      minimo ~~< temp <~~ maximo

# El tipo char

El tipo `char` permite representar el conjunto de 256 letras mayúsculas y minúsculas, dígitos y otros caracteres.

Cada caracter está representado internamente como un número entero denotando su código ASCII.

Los literales de tipo de char se denotan entre apóstrofes, por ej. 'A', 'H', ..., 'a', 'c', ..., '"', '@', ..., '0', '¿', '2', ...

[illegible]

# El tipo char

tIPOS

American Standard Code for Information Interchange

Está formado por 256 símbolos, asociados a su posición en la tabla.

|     |   |     |   |     |   |     |   |     |   |     |   |     |    |     |   |     |   |     |   |
|-----|---|-----|---|-----|---|-----|---|-----|---|-----|---|-----|----|-----|---|-----|---|-----|---|
|     |   |     |   | 32  |   | 33  | ! | 34  | " | 35  | # | 36  | \$ | 37  | % | 38  | & | 39  | ' |
| 40  | ( | 41  | ) | 42  | * | 43  | + | 44  | , | 45  | - | 46  | .  | 47  |   | 48  | 0 | 49  | 1 |
| 50  | 2 | 51  | 3 | 52  | 4 | 53  | 5 | 54  | 6 | 55  | 7 | 56  | 8  | 57  | 9 | 58  | : | 59  | ; |
| 60  | < | 61  | = | 62  | > | 63  | ? | 64  | @ | 65  | A | 66  | B  | 67  | C | 68  | D | 69  | E |
| 70  | F | 71  | G | 72  | H | 73  | I | 74  | J | 75  | K | 76  | L  | 77  | M | 78  | N | 79  | O |
| 80  | P | 81  | Q | 82  | R | 83  | S | 84  | T | 85  | U | 86  | V  | 87  | W | 88  | X | 89  | Y |
| 90  | Z | 91  | [ | 92  | \ | 93  | ] | 94  | ^ | 95  | _ | 96  | `  | 97  | a | 98  | b | 99  | c |
| 100 | d | 101 | e | 102 | f | 103 | g | 104 | h | 105 | i | 106 | j  | 107 | k | 108 | l | 109 | m |
| 110 | n | 111 | o | 112 | p | 113 | q | 114 | r | 115 | s | 116 | t  | 117 | u | 118 | v | 119 | w |
| 120 | x | 121 | y | 122 | z | 123 | { | 124 |   | 125 | } | 126 | ~  | 127 | ! | 128 | Ç | 129 | ü |

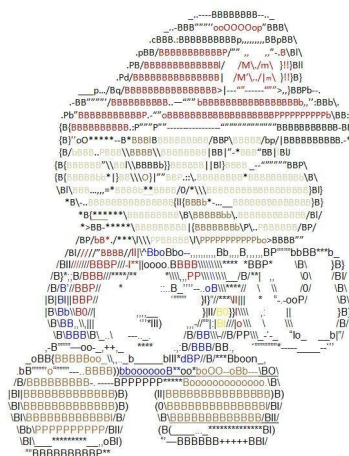
# El tipo char

tIPOS

Los operadores **pred** y **succ** aplicados a un operando de tipo char computan un valor de tipo char.

El operador **chr** aplicado sobre un operando de tipo entero computa un valor de tipo char.

El operador **ord** aplicado sobre un operando de tipo char computa un valor de tipo integer



**pred('z') → 'y'** *Anterior a 'z'*

**succ('C') → 'D'** *Siguiente a 'C'*

**chr(35) → '#'** *Caracter en la posición 35*

**ord('M') → 77** *Posición del caracter 'M'*



**EL TIPO QUE DEFINIMOS  
PARA UNA VARIABLE  
DETERMINA LOS VALORES  
QUE ESA VARIABLE PUEDE TOMAR,  
Y LAS OPERACIONES  
EN LAS QUE PUEDE PARTICIPAR.**



# Variables



Es fundamental  
diferenciar entre la versión  
estática y la dinámica  
de un programa

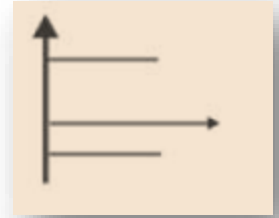
En la **declaración** de una variable se establece:

- **Nombre:** ¿cualquier? identificador no reservado.
- **Tipo:** determina el conjunto de valores que puede tomar y en qué operaciones puede usarse.

En **ejecución** una variable tendrá:

- **Locación:** lugar en memoria en el cual se almacena el valor.
- **Valor:** contenido que puede ir cambiando y que pertenece al conjunto de valores establecido por el tipo.

# Constantes



Queda ligada a un **identificador** mediante una declaración.

```
const
  PI = 3.1415;
  fin_texto = ' ';

var x: real; Ch: char;
...
begin
  x:= 2 * PI;
  Ch:='*';
  fin:= (Ch = fin_texto)
  ...
```

**NO!!**

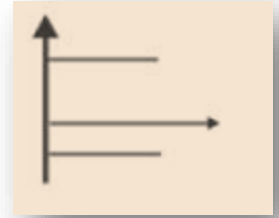


```
1  program constante;
.  const a=10;
.
.  begin
.  a:=11;
5
6  end.
```

Ventana de Mensajes

❗ caseSimb.lpr(5,3) Error: Variable identifier expected  
💣 caseSimb.lpr(7) Fatal: There were 1 errors compiling module, stopping

# Constantes



```
program saludos;  
const SALUDO='HOLA';
```

```
begin  
    writeln(SALUDO);  
    writeln(SALUDO);  
    writeln(SALUDO);  
    writeln(SALUDO);  
    writeln(SALUDO);  
    readln;  
end.
```

La ventaja principal es que si queremos modificar el saludo, lo hacemos en un único lugar.

# Constantes

Es fundamental diferenciar  
ente la versión estática  
y la dinámica de un programa

En la **declaración** de una constante se establece:

- **Nombre:** ¿cualquier? identificador no reservado.
- **VALOR:** lo tendrá a lo largo de todo el programa.

El compilador **INFIERE el tipo** de la constante a partir del dominio al que pertenece el valor ligado a la misma.

Al traducir el código fuente, el compilador reemplaza cada aparición del identificador de constante por su valor.

En **ejecución** la constante no es almacenada en memoria,  
y no puede cambiar su valor.

El **tipo de una variable** se especifica en su declaración:

var

a: integer;

b: real;

El **tipo de una constante** es inferido por el compilador a partir de su valor:

const

maximo = 100;

descuento = 12.5;

Establecer los tipos de los datos  
permite realizar chequeos en compilación  
que aseguran (*almost always*)  
el correcto funcionamiento de las operaciones.

```

1  program tipos;
.  const a=1.5;
.  var b: integer;
.  begin
5  [ b:=a;
6  end.

```

No puedo asignar un valor real  
a una variable de tipo entero.

8      Modificado      INS      project1.lpr

Ventana de Mensajes

❗ project1.lpr(5,8) Error: Incompatible types: got "Single" expected "LongInt"  
 🛑 project1.lpr(7) Fatal: There were 1 errors compiling module, stopping

Establecer los tipos de los datos  
permite realizar chequeos en compilación  
que aseguran (*almost always*)  
el correcto funcionamiento de las operaciones.

```
program chequeos;
var a: integer;
    b: real;
    c: char;
begin
  a := 10;
  b := 3.4;
  c := 'M';
  b := a + b;
  a := a + b;
  b := b + c;
end.
```



Compilation failed due to following error(s).

```
Free Pascal Compiler version 3.2.2+dfsg-32 [2024/01/05] for x86_64
Copyright (c) 1993-2021 by Florian Klaempfl and others
Target OS: Linux for x86-64
Compiling main.pas
main.pas(18,12) Error: Incompatible types: got "Real" expected "SmallInt"
main.pas(19,12) Error: Operator is not overloaded: "Real" + "Char"
main.pas(22) Fatal: There were 2 errors compiling module, stopping
Fatal: Compilation aborted
Error: /usr/bin/ppcx64 returned an error exitcode
```



Establecer los tipos de los datos  
permite realizar chequeos en compilación  
que aseguran (*almost always*)  
el correcto funcionamiento de las operaciones.

```

1  program conversionDeTipo;
.  var a: integer; b: real;
.  begin
.    a:=10;
5  b:=a
6  end.
```



En este caso se permite “mezclarlos”  
ya que Pascal realizará una **conversión implícita**  
de tipo entero a tipo real  
para el valor de la variable a antes de asignarlo.

# Verificación de un algoritmo

¿Qué significa que un algoritmo sea correcto?

Para cualquier valor de los datos de entrada,  
devuelve los datos de salida esperados.



# Verificación de un algoritmo

¿Cómo se sabe si un algoritmo  
es correcto?

En forma **estática** (teórica) es muy difícil...

En forma **dinámica**  
se puede testear su comportamiento.

**No es una demostración formal  
de correctitud!**

Condición necesaria  
pero no suficiente



# Verificación de un algoritmo mediante trazas

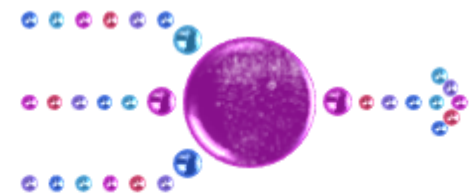


Una **traza** nos permite analizar el comportamiento de los datos a lo largo de la “ejecución”.

Simulamos la EJECUCION del algoritmo para **ciertos datos de entrada**, y vamos registrando los cambios en todos los datos.

En forma gráfica, podemos usar una tabla:

- Columnas → asociadas a los datos
- Filas → cambios en los valores



# Importancia de las trazas

¡FUNDAMENTAL!

Realizar trazas para valores  
**representativos** de los datos de entrada.

- Sirven para depurar un algoritmo pero no son una técnica de demostración de correctitud.
- Sólo aseguran que un algoritmo funciona de la manera esperada para los valores de los datos de entrada usados.



# La traza de una secuencia de asignaciones

```
PROGRAM abc;  
VAR a, b, c: integer;  
BEGIN  
  a := 1;  
  b := a+1;  
  c := b*2;  
  a := b+c;  
  writeln(a, ' ', b, ' ', c);  
END.
```

| a | b | c |
|---|---|---|
|   |   |   |



# La traza de una secuencia de asignaciones

```
PROGRAM abc;  
VAR a, b, c: integer;  
BEGIN  
  a := 1;  
  b := a+1;  
  c := b*2;  
  a := b+c;  
  writeln(a, ' ', b, ' ', c);  
END.
```

| a | b | c |
|---|---|---|
| 1 |   |   |



# La traza de una secuencia de asignaciones

```
PROGRAM abc;  
VAR a, b, c: integer;  
BEGIN  
  a := 1;  
  b := a+1;  
  c := b*2;  
  a := b+c;  
  writeln(a, ' ', b, ' ', c);  
END.
```

| a | b | c |
|---|---|---|
| 1 | 2 |   |





# La traza de una secuencia de asignaciones

```
PROGRAM abc;  
VAR a, b, c: integer;  
BEGIN  
  a := 1;  
  b := a+1;  
  c := b*2;  
  a := b+c;  
  writeln(a, ' ', b, ' ', c);  
END.
```

| a | b | c |
|---|---|---|
| 1 | 2 | 4 |



# La traza de una secuencia de asignaciones

```
PROGRAM abc;  
VAR a, b, c: integer;  
BEGIN  
  a := 1;  
  b := a+1;  
  c := b*2;  
  a := b+c;  
  writeln(a, ' ', b, ' ', c);  
END.
```

| a | b | c |
|---|---|---|
| 6 | 2 | 4 |

6 2 4

La asignación BORRA el valor anterior (si hubiera uno) y almacena el nuevo.

# La traza de una secuencia de asignaciones

```
PROGRAM abc2;  
VAR a, b, c: integer;  
BEGIN  
  a := 1;  
  b := a+1;  
  c := b*2;  
  a := a+1;  
  writeln(a, ' ', b, ' ', c);  
END.
```



La misma variable puede aparecer a la izquierda y a la derecha de una asignación. DOS PASOS: Primero se computa la expresión a la derecha del “:=”; luego se almacena el valor computado.

# La traza de una secuencia de asignaciones

```
PROGRAM abc3;  
VAR a, b, c: integer;
```

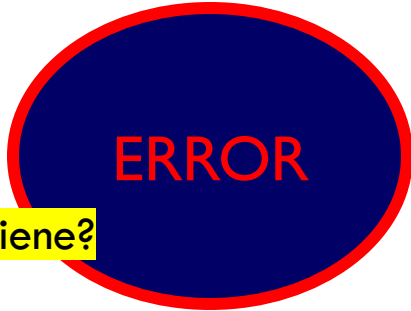
```
BEGIN
```

```
  b := a + 1;
```

```
  c := b * 2;
```

```
  writeln(a, ' ', b, ' ', c);
```

```
END.
```



ERROR

¿Qué valor tiene?

Formas de DAR VALOR  
a una variable:

- read(variable)
- Asignacion  
variable ← (....)

La variable **a** aparece en una expresión sin haber sido inicializada.

**NO DEBEMOS “usar” una variable que no está inicializada.**

# La memoria

*Dispositivo de almacenamiento de la computadora.*

Los valores de las **variables**  
*se almacenan en celdas de memoria*  
durante la **EJECUCION** del programa.

Una declaración como:

```
var a, b, c: integer;
```

Ocupa **EN EJECUCION** tres celdas de memoria asociadas a los nombres a, b y c.

Cuando el programa termina las celdas se *liberan*, y en la próxima ejecución probablemente se ocupen otras.

ACCIONES para inicializar o modificar el valor de un dato

- La instrucción *readln(a)* (LECTURA) modifica el valor almacenado en la celda ligada a la variable *a* dándole el valor ingresado por el usuario.
- La instrucción *a := b+c* (ASIGNACION) modifica el valor almacenado en la celda ligada a la variable *a*, dándole el valor obtenido de evaluar la expresión de la derecha (en este caso,  $b+c$ ).

*a := b+c*

# La traza de una secuencia de asignaciones



```
PROGRAM intercambio;
```

```
VAR a, b, aux: integer;
```

```
BEGIN
```

```
  write('Ingrese dos valores, primero el de a y luego el de b:');
```

```
  read(a, b);
```

```
  aux := a;
```

```
  a := b;
```

```
  b := aux;
```

```
  writeln('Intercambiados son: a= ', a, ' y b= ', b);
```

```
END.
```

¿Es equivalente?

aux := a;

b := aux;

a := b;



# Problema general

## Código fuente – visión estática de nuestro programa

```
1
2 program Resumiendo;
3 // declaracion de variables
4 var a: integer;
5     b, c: real;
6 begin
7     write('Ingrese un valor para el dato a: '); // primitiva para mostrar información en la consola
8     read(a); // primitiva de lectura para obtener un valor de la consola
9     b := a * 2.3; // primitiva de asignación para dar un valor a una variable
10    c := a + b; // primitiva de asignación para dar un valor a una variable
11    writeln('RESULTADO: '); // primitiva para mostrar información en la consola
12    writeln(' Para a valiendo ', a, ' b vale ', b:0:2, ' y c vale ', c:0:2);
13    { primitiva para mostrar información en la consola }
14 end.
```

## Resolución de instancias particulares del problema

### Ejecuciones – visión dinámica de nuestro programa

```
Ingrese un valor para el dato a:  7
RESULTADO:
Para a valiendo 7 b vale 16.10 y c vale 23.10
```

```
Ingrese un valor para el dato a:  21
RESULTADO:
Para a valiendo 21 b vale 48.30 y c vale 69.30
```

```
Ingrese un valor para el dato a:  3
RESULTADO:
Para a valiendo 3 b vale 6.90 y c vale 9.90
```



# La traza de una secuencia de asignaciones



```
PROGRAM abc4;  
VAR a, b, c: integer;  
BEGIN  
  a := 1;  
  b := a + 1;  
  c := b * 2;  
  c := c + a + b;  
  a := b * a;  
  a := a + 1;  
  writeln (a, ' ', b, ' ', c);  
END.
```

Modifique el programa para que la variable "a" sea dato de entrada. Realice 2 trazas:

- 1) Asumiendo que el usuario ingresa un 10
- 2) Asumiendo que el usuario ingresa un -1

*La misma variable puede aparecer a la izquierda y a la derecha de una asignación.*

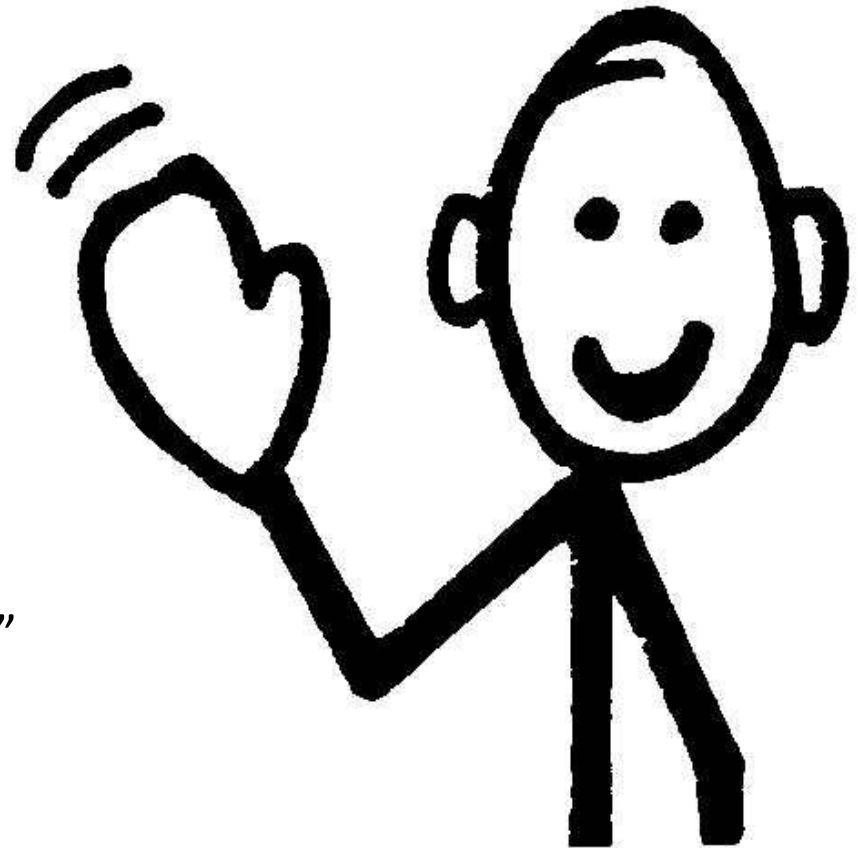
*DOS PASOS: 1) Se evalúa la expresión en el lado derecho*

*2) Se almacena el valor obtenido en la variable del lado izquierdo.*

**CHAU!**  
**HASTA LA PRÓXIMA CLASE**



DESCARGAR de  
“MOODLE → RECURSOS ADICIONALES”  
el archivo de errores de compilación  
y ver los videos de expresiones.



# CICLO DE PROGRAMACIÓN



# Elementos de Pascal

Problema: Calcular y mostrar el promedio de tres notas ingresadas por el usuario.

Algoritmo PromedioNotas

DE: n1, n2, n3 (enteros)

DS: promedio (real)

promedio  $\leftarrow (n1+n2+n3)/3$

Calcular  
promedio  
de 3 notas

# Elementos de Pascal

```
program promedioNotas;  
var n1, n2, n3: integer; {DE}  
    promedio: real; {DS}  
begin  
    write ('Ingrese las tres notas');  
    readln (n1, n2, n3);  
    promedio := (n1+n2+n3)/3;  
    write ('El promedio de ', n1, n2, n3);  
    writeln(' es ', promedio);  
end.
```

Calcular promedio  
de 3 notas

En esta versión se reflejan  
claramente tanto los datos de  
entrada como el dato de salida  
(**promedio**).

Le damos un nombre al dato de  
salida a diferencia de la diapo  
anterior.

**Es más prolijo y más legible.**